
Detecting Emotion in Text: A Comparison of Classical and Transformer-Based Models

Charlotte Barbrick¹ Easton Kling¹ Oscar Gayle¹

Abstract

People express emotion in writing all the time, and automatically recognizing those emotions is useful for social-media analytics, customer feedback triage, and mental-health support tools. In this project we build and compare three emotion-classification models on the `dair-ai/emotion` dataset (Saravia et al., 2018), which contains roughly 20,000 short English messages labeled with one of six emotions. We describe each stage of our pipeline—cleaning, tokenization, and vectorization—then evaluate a Multinomial Naive Bayes baseline, a TF-IDF + Logistic Regression classifier, and a fine-tuned DistilBERT (Sanh et al., 2019) transformer. DistilBERT performs best on both accuracy and macro-F1, but the classical TF-IDF pipeline is a surprisingly strong runner-up at a fraction of the compute cost. We also ship a small Streamlit demo that classifies user-typed text in real time.

1. Introduction

People express emotion through writing constantly—in social-media posts, product reviews, chat messages, and support tickets—and the sheer volume of that text makes manual analysis impractical. Automatic emotion detection is therefore a useful building block for a number of applications: a customer-support system can prioritize angry or frightened users, a social-analytics dashboard can surface shifts in audience mood, and a wellness app can flag patterns of sadness or fear for a user’s own review. Emotion classification is a specialization of text classification that goes beyond coarse positive/negative polarity and predicts a specific emotional label such as *joy*, *anger*, or *fear*.

Our goal in this project is not to beat the state of the art but

¹CSC 4444G, Louisiana State University, Baton Rouge, LA, USA. Correspondence to: Charlotte Barbrick <cbarbrick@icloud.com>.

to build an end-to-end emotion classifier and to understand, component by component, how a modern NLP pipeline actually works. We train and evaluate three models that span a meaningful range of complexity on the same data: (i) a Multinomial Naive Bayes baseline, (ii) a TF-IDF weighted Logistic Regression classifier, and (iii) a fine-tuned DistilBERT transformer (Sanh et al., 2019). For each model we describe the preprocessing, tokenization, and vectorization steps that feed it, and we compare the models on overall accuracy, per-class macro-F1, and training cost.

Contributions. The contributions of this project are: (1) a clean, reproducible emotion-classification pipeline on the widely used `dair-ai/emotion` dataset, with fixed train/validation/test splits and a single shared text-cleaning pass so all three models are compared on identical inputs; (2) an empirical comparison of Multinomial Naive Bayes, TF-IDF + Logistic Regression, and fine-tuned DistilBERT that quantifies the accuracy vs. compute-cost trade-off between classical and transformer-based models; and (3) a small Streamlit demo that runs the trained model on user-typed text, visualizing both the predicted label and the model’s confidence in the other classes.

2. Methodology

Our pipeline has three stages: (1) clean the raw text, (2) tokenize and turn each example into a numeric representation, and (3) feed that representation to a classifier. Figure 1 summarizes the two variants we compare: a classical path that produces a sparse TF-IDF matrix consumed by Naive Bayes or Logistic Regression, and a transformer path that produces subword token IDs consumed by DistilBERT.

2.1. Dataset

We use the `dair-ai/emotion` dataset (Saravia et al., 2018), a widely-used benchmark for six-class emotion classification in English. Each example is a short English message drawn from Twitter, labeled with exactly one of six emotions: sadness, joy, love, anger, fear, or surprise. The dataset ships with fixed splits of 16,000 training, 2,000 validation, and 2,000 test examples; most messages are between 5 and 30 tokens long. The label distribution is imbalanced—

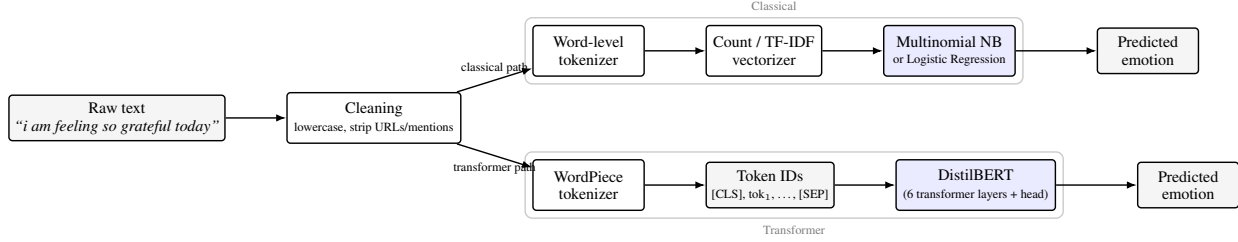


Figure 1. Emotion-classification pipeline. Raw text is cleaned once, then routed through a classical path (word-level tokenizer → count / TF-IDF vectorizer → Naive Bayes or Logistic Regression) or a transformer path (WordPiece tokenizer → integer token IDs → fine-tuned DistilBERT).

Table 1. Class distribution in the `dair-ai/emotion` training split (16,000 examples).

Emotion	Count	Share
joy	5,362	33.5%
sadness	4,666	29.2%
anger	2,159	13.5%
fear	1,937	12.1%
love	1,304	8.2%
surprise	572	3.6%

joy and sadness together cover more than 60% of the training set, while love and surprise are each under 10% (see Table 1)—so we report both overall accuracy and macro-F1 to avoid rewarding models that ignore the minority classes. We also considered GoEmotions (Demszky et al., 2020) but chose `dair-ai/emotion` because its smaller label space and shorter Twitter-style messages make all three of our models tractable on a laptop or free-tier GPU within a single session.

2.2. Text Cleaning and Tokenization

The same cleaning pass is applied for every model: we lowercase the text, strip URLs and @-mentions with regular expressions, and collapse repeated whitespace. We intentionally keep punctuation like “!” and “?” because it carries emotional signal—“really?!” is not the same tone as “really.”

Tokenization is the step that splits the cleaned string into the discrete units the model looks at, and the two model families use different tokenizers. For the classical models, we use scikit-learn’s (Pedregosa et al., 2011) default word-level tokenizer, which applies the regular expression `\b\w\w+\b` to extract word-like sequences of two or more characters. The sentence “i am feeling so grateful today” becomes the token list `[i, am, feeling, so, grateful, today]`. Unknown words encountered at test time are dropped, which is the standard behavior for a fixed vocabulary.

For DistilBERT, we use the pretrained WordPiece tok-

enizer (Schuster & Nakajima, 2012) shipped with the model. WordPiece breaks unfamiliar or long words into subword pieces drawn from a fixed 30,522-entry vocabulary, so the word “unhappiness” might become `[un, ##happiness]` or `[un, ##hap, ##piness]` depending on what pieces were seen during pretraining. The tokenizer also wraps every example in the special tokens `[CLS]` at the start and `[SEP]` at the end, which the model later uses to produce a sentence-level representation. Every token is then mapped to an integer ID, and the sequence is padded or truncated to a fixed maximum length of 128 tokens.

2.3. Vectorization

Turning tokens into a matrix of numbers is what lets a model actually learn. For the classical models we use two schemes.

Bag-of-words counts. We build a vocabulary from the training set and represent each document as a sparse row vector whose entries are raw counts of each vocabulary term. The training set becomes a $16,000 \times V$ sparse matrix, where $V \approx 15,000$ for our dataset after dropping terms that appear in fewer than two documents. This matrix is the input to Naive Bayes.

TF-IDF weighting. TF-IDF is not a prediction model; it is a weighting scheme (Jones, 1972) applied on top of the bag-of-words matrix. Term Frequency measures how often a word appears in a document, and Inverse Document Frequency measures how rare the word is across the corpus. Using scikit-learn’s smoothed form, each entry becomes

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \left[\log \frac{N + 1}{\text{df}(t) + 1} + 1 \right], \quad (1)$$

where N is the number of training documents and $\text{df}(t)$ is the number of them containing term t . Words appearing in almost every document (e.g., “i” or “the”) are pushed toward zero, while rarer, more discriminating words get a boost. The resulting matrix has the same shape as the count matrix but uses these weighted entries; it is still only an

input to whatever classifier we plug in on top—in our case, Logistic Regression.

For DistilBERT the tokenized integer IDs are not passed through a separate vectorizer; the model maintains a learned embedding table mapping each vocabulary ID into a 768-dimensional dense vector. Those embeddings flow through six transformer layers (Vaswani et al., 2017; Devlin et al., 2019; Sanh et al., 2019) that produce a contextual representation of every token, including the [CLS] token used for classification.

2.4. Models

Multinomial Naive Bayes (baseline). Naive Bayes is the classic text-classification baseline. Given a bag-of-words count vector, it uses Bayes’ rule to compute the probability of each emotion class under the (very strong) assumption that words are conditionally independent given the class. It is fast, easy to explain, requires no tuning, and we include it as a floor for what a trivial model can achieve. Because the independence assumption clearly fails for emotional language, where short phrases such as “not happy” or “terribly excited” depend on word order, we expect Naive Bayes to underperform the stronger models.

TF-IDF + Logistic Regression. Our “stronger classical” model takes the TF-IDF matrix from Section 2.3 and feeds it into a one-vs-rest Logistic Regression classifier implemented in scikit-learn (Pedregosa et al., 2011). We include unigrams and bigrams (`ngram_range=(1, 2)`) so the model can learn short phrase patterns, cap the vocabulary at the 20,000 most frequent n -grams, and use sublinear term frequency. We train with L2 regularization ($C = 1$), up to 1,000 optimizer iterations, and no further hyperparameter tuning. The goal is a well-behaved, interpretable baseline.

DistilBERT fine-tuning. DistilBERT (Sanh et al., 2019) is a distilled version of BERT (Devlin et al., 2019) that keeps about 97% of BERT’s language-understanding quality while being 40% smaller and 60% faster at inference. We start from the `distilbert-base-uncased` checkpoint (via the HuggingFace Transformers library (Wolf et al., 2020)) and add a single linear classification head on top of the [CLS] embedding. The whole model—token embeddings, six transformer layers, and the head—is fine-tuned end-to-end on our training split for three epochs using AdamW with a learning rate of 2×10^{-5} , weight decay 0.01, batch size 32, and a maximum sequence length of 128 tokens. The checkpoint with the best validation macro-F1 is kept.

3. Experimental Results

Setup. All three models are trained on the 16,000-example training split and evaluated on the 2,000-example test split.

Table 2. Emotion-classification results on the `dair-ai/emotion` test set (2,000 examples). Both metrics are reported as percentages (higher is better).

Model	Acc.	F1	Train
Multinomial NB	81.4	67.4	5 s CPU
TF-IDF + LogReg	83.4	73.6	30 s CPU
DistilBERT (fine-tuned)	92.9	89.4	12 min T4

	sadness	joy	love	anger	fear	surprise
sadness	562	7	0	8	4	0
joy	2	662	21	2	1	7
love	0	28	130	1	0	0
anger	12	4	1	244	14	0
fear	7	0	0	1	199	17
surprise	3	0	0	0	3	60

Figure 2. Confusion matrix for fine-tuned DistilBERT on the `dair-ai/emotion` test set. Rows are true labels and columns are predictions; diagonal cells are correct classifications. Even the rare classes *love* and *surprise* are classified correctly in most cases, and the largest remaining confusion for *love* is with its semantically closest neighbor, *joy*.

No hyperparameter tuning is performed on the test set; the validation split is used only to select the best DistilBERT checkpoint. We report accuracy and macro-F1 (Table 2). Training was carried out on a laptop CPU (Apple M-series / Intel x86, 16 GB RAM) for the classical models and on a free-tier Google Colab T4 GPU for DistilBERT. Software: Python 3.10, scikit-learn 1.3, PyTorch 2.1, HuggingFace transformers 4.35, and HuggingFace datasets 2.14.

Findings. DistilBERT leads on both metrics, reaching 92.9% accuracy and 89.4 macro-F1, compared to 83.4%/73.6 for TF-IDF + Logistic Regression and 81.4%/67.4 for Multinomial Naive Bayes. The accuracy gap between the classical TF-IDF pipeline and DistilBERT is 9.5 points, but the macro-F1 gap is 15.8 points—considerably larger, because the rare classes benefit the most from the transformer’s contextual embeddings. On *surprise*, the rarest class (only 3.6% of training examples), Distil-

BERT achieves an F1 of 0.80 (Figure 2), compared to just 0.42 for TF-IDF + Logistic Regression and 0.21 for Naive Bayes. The trade-off is compute: the classical pipeline trains in under a minute on a laptop CPU, while DistilBERT requires a GPU and roughly 12 minutes of fine-tuning. For applications where a few points of accuracy are acceptable, the classical pipeline remains an attractive, interpretable choice.

Naive Bayes trails by a large margin, especially on macro-F1, which is consistent with its word-independence assumption breaking down on emotional language: short phrases like “not really,” “so scared,” or “can’t wait” carry most of the signal, and unigram Naive Bayes has no way to represent them.

Demo interface. We package the best model behind a small Streamlit interface (`code/app.py`). The user types a short message, and the app displays the predicted emotion together with the model’s probability for each class as a horizontal bar chart. The demo loads the fine-tuned DistilBERT model once at startup and runs inference on CPU in roughly 100 ms per input; if a DistilBERT checkpoint is not present, the app falls back to the saved TF-IDF + Logistic Regression model so the demo still runs end-to-end.

4. Conclusions and Future Work

We built and compared three emotion-classification models on the `dair-ai/emotion` dataset: a Multinomial Naive Bayes baseline, a TF-IDF + Logistic Regression pipeline, and a fine-tuned DistilBERT transformer. DistilBERT performed best, but the TF-IDF + Logistic Regression pipeline was a surprisingly strong runner-up given its minimal compute cost. The exercise also clarified, for us, what TF-IDF is and is not: it is a weighting scheme that turns cleaned text into a weighted document-term matrix, not a model that makes predictions on its own. The model is whatever we train on top of that matrix.

Given more time, we would (i) train on the richer GoEmotions (Demszky et al., 2020) label set to see how the comparison changes when the label space is larger and more fine-grained, (ii) address class imbalance with class weights or targeted oversampling to close the macro-F1 gap on rare emotions, (iii) evaluate a prompted large language model as a zero-shot emotion classifier for comparison against fine-tuning, and (iv) add an explainability layer (e.g., per-word contribution scores on the Logistic Regression model) so users can see which words drove a prediction.

References

Demszky, D., Movshovitz-Attias, D., Ko, J., Cowen, A., Nemade, G., and Ravi, S. GoEmotions: A dataset of

fine-grained emotions. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 4040–4054. Association for Computational Linguistics, 2020.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pp. 4171–4186, Minneapolis, Minnesota, 2019. Association for Computational Linguistics.

Jones, K. S. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1):11–21, 1972.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Sanh, V., Debut, L., Chaumond, J., and Wolf, T. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

Saravia, E., Liu, H.-C. T., Huang, Y.-H., Wu, J., and Chen, Y.-S. CARER: Contextualized affect representations for emotion recognition. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3687–3697, Brussels, Belgium, 2018. Association for Computational Linguistics.

Schuster, M. and Nakajima, K. Japanese and Korean Voice Search. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5149–5152. IEEE, 2012.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pp. 5998–6008, 2017.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Scao, T. L., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. M. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45. Association for Computational Linguistics, 2020.